



CLARUSWAY

WAY TO REINVENT YOURSELF



clarusway



clarusway



clarusway



clarusway



clarusway

Interface

Lesson:
JAVA Chapter 27

Interface

- **Interface** bir class değildir. Interface **interface'dir**. Interface'ler somutlaştırılmaz (can not be instantiated) Interface'de obje create edilemez.
- Java'da arayüzler (**Interfaces**), yapısal olarak **Abstract class'lara** benzemektedirler. Java'da **interface** ile **Abstract class'lara** arasındaki temel fark; **Abstract class'larda** her metodu override etmek zorunda değildir.
- Java'da **interface'de** ise tanımladığımız her method, concrete class'larda mutlaka override edilmelidir. Java'da **interface'ler**, class'ların **bir methodda action'un nasıl yapılacağını değil ne yapılacağını belirler**.
- **Interface** kendisinden üretilen sınıflar için bir kılavuz, yol gösterici veya gerçekleştirilmesi gereken görevler bütünü **to do list** denilebilir.
- **Interface** içinde sadece kendisinden türeyen sınıfların içeriğini doldurmak zorunda olduğu, body'si olmayan method'ların oluşturulduğu bir yapıdır.
- Java **Interface** ayrıca **IS-A** ilişkisini temsil eder .
- **Interface** değişkenleri genellikle final ve static olur çağırılan sınıf değişkenleri direkt kullanır değerlerini değiştiremez.



Interface

Java'da Arayüzler'in (Interfaces) Avantajları

- **interface'ler** aracılığıyla programda çoklu kalıtım uygulayabiliriz.
- **interface'ler**, karmaşık tasarımları parçalama ve nesneler arasındaki bağımlılıkları temizleme işlevi görür.
- **interface'ler**, uygulamayı gevşek (loose coupling) bir şekilde birleştirir.
- **interface**'deki tüm methodları implement etmek zorunda bırakır.



Interface

```
1 interface kitap {
2
3     int syf = 189;
4
5     void goster(); //gosterilmesi için soyut method create edildi.
6 }
7 //kitap interface'i implements eden bir class create edildi
8 class Kitabım implements kitap {
9
10     @Override //interface'in özellikleri uygulandı.
11     public void goster() {
12         System.out.println("Fareler ve insanlar");
13     }
14
15     public static void main (String [] args) {
16         Kitabım ktp1 = new Kitabım(); //method call için obj. create edildi.
17
18         ktp1.goster();
19         System.out.println("Sayfa sayısı = " +syf);
20     }
21 }
```



Interface

- Interface kendisinden üretilen sınıflar için bir kılavuz, yol gösterici veya gerçekleştirilmesi gereken görevler bütünü **to do list** denilebilir.
- Bir class birden fazla class'a extend edilemez ama birden fazla interface'e implement edilebilir.

Fen Bilimler

```
matematik();
fizik();
kimya();
biyoloji(),
```

Sosyal Bilimler

```
edebiyat();
tarih();
cografya();
felsefe(),
```

Spor

```
futbol();
basketbol();
voleybol();
tenis(),
```

Güzel Sanatlar

```
resim();
müzik();
dans();
```

Proje

```
..... ();
..... ();
..... ();
```

Tez

```
..... ();
..... ();
..... ();
```

Staj

```
..... ();
..... ();
..... ();
```

Akademik Ders

Sosyal Faaliyet

Ödev

implements

implements

implements

```
Akademik ders
Sosyal faaliyet
ödev
```

Öğrenci



Interface vs. Abstract Classes

- **Abstract** yapılar class'dır. ← → **Interface** yapılar class değildir.
- **Abstract** Class'lar abstract ve concrete methodlar bulundurabilir. ← → **Interface** sadece abstract method bulundurabilir ama concrete method bulunduramaz.
- **Abstract** Class'lar kısmi abstraction sağlar. ← → **Interface** tam abstraction sağlar.
- **Abstract** Class'lar çoklu inheritance desteklemez. ← → **Interface** çoklu inheritance destekler.
- **Abstract** Class'lar constructor bulundurabilir. ← → **Interface** constructor bulunduramaz.
- **Abstract** Class'lardaki üyeler «public» olması gerekmez. ← → **Interface** üyeleri public olmalıdır.
- **Abstract** Class daha hızlıdır. ← → **Interface** daha yavaştır.
- **Abstract** Class'larda abstract methodlar static tanımlanabilir. ← → **Interface** methodlar static tanımlanamaz.



Interface

```
3 public interface Clarusway {  
4  
5     int kursUcret = 1000;  
6     public int ogrcSayisi = 1000;  
7     public static int ogrtSayisi = 11;  
8     public static final int kursSuresi = 9;  
9  
10    void kursKayit();  
11  
12    public void projeKayit();  
13  
14    abstract int kursOdeme();  
15  
16    public abstract void kursKayitSil();  
17 }
```

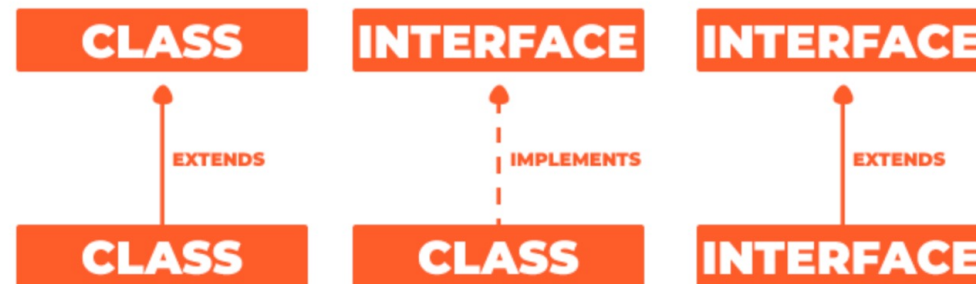
- **Interface** variable'ları mutlaka **public**, **static**, ve **final** tanımlanmalı. Private veya protected tanımlanan variable'lar CTE verir.
- **Interface** variable'ları mutlaka **initialize** (ilk değer ataması) edilmeli, aksi takdirde CTE verir.
- **Interface'de sadece abstract public method'lar** tanımlanabilir.
- **Interface** methodlarının **return type'ları** farklı olabilir.



Interface'ler için Inheritance Kuralları

* Interface'lerde inheritance yapmak için **implements** keyword'u kullanılır.

JAVA INTERFACE KULLANIMI



* Bir class birden fazla Interface'e **implements** edilebilir.

```
public class QaTester implements Clarusway , Bootcamp, ItEgitim {  
  
}
```



Interface'ler için Inheritance Kuralları

* TRICK :)

Birden fazla Interface'i **implements** ile inherit edildiğinde aynı name sahip variable **interface name** ile call edilir.

```
public class QaTester implements Clarusway, Bootcamp, ItEgitim {  
  
    public static void main(String[] args) {  
        System.out.println("offer sayıları : " + ItEgitim.OFFERSAYISI); // 1001001  
        System.out.println("offer sayıları : " + Bootcamp.OFFERSAYISI); // 10101  
        System.out.println("offer sayıları : " + Clarusway.OFFERSAYISI); // 101  
    }  
}
```



Interface'ler için Inheritance Kuralları

* TRICK :)

Birden fazla Interface'i **implements** ile inherit edildiğinde aynı name sahip methodlar zorunlu olarak override edileceği için hangi **interface'den** call edildiğinin önemi yoktur .

```
public interface Clarusway {  
    int OFFERSAYISI = 101;  
  
    void kursKayit();  
}
```

```
public interface Bootcamp {  
    int OFFERSAYISI = 10101;  
  
    void kursKayit();  
}
```

```
public interface ItEgitim {  
    int OFFERSAYISI = 1001001;  
  
    void kursKayit();  
}
```

```
public class QaTester implements Clarusway, Bootcamp, ItEgitim {  
  
    @Override  
    public void kursKayit() {  
        System.out.println("madem geldin dünyaya; otur çalış JAVA'ya :) ");  
    }  
}
```



Interface'ler için Inheritance Kuralları

* TRICK :)

Method name aynı fakat return type'ı farklı methodlara sahip Interface'ler aynı class'dan inherit edilemez CTE verir.

```
public interface Clarusway {  
    int OFFERSAYISI = 101;  
  
    public void kursKayit();  
}
```

```
public interface Bootcamp {  
    int OFFERSAYISI = 10101;  
  
    public int kursKayit();  
}
```

```
public class QaTester implements Clarusway, Bootcamp, ItEgitim {  
  
    @Override  
    public void kursKayit() {  
        System.out.println("madem geldin dünyaya; otur çalış JAVA'ya :) ");  
    }  
}
```



Interface'ler için Inheritance Kuralları

TRICK :)

- JAVA 8'e kadar interface'de "body'li concrete method" tanımlanamazdı. JAVA 8 ile bu özellik update edildi.
- Method access modifier public olduğu halde "default" keyword method type olarak kullanılmaz ise CTE verir.

```
public interface Clarusway {  
  
    public void kursKayit();  
  
    public default String offer() {  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```

```
public interface Clarusway {  
  
    public void kursKayit();  
  
    public String offer() {  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```



Interface'ler için Inheritance Kuralları

TRICK :)

- Concrete method'da **"default"** keyword yerine **"static"** keyword de kullanılabilir ama hem **"default"** hem de **"static"** keyword birlikte **kullanılamaz.**
- Buradaki **"static"** keyword interface özel bir keyword'dur aynı package'deki başka class'dan class name ile method call **edilemez.**
- Default veya static keyword'u ile tanımlanan method'lar concrete olduğu için override yapılması **zorunlu değildir.**

```
public interface Clarusway {  
  
    public void kursKayit(); // abstract method  
  
    public static String offer() { // concrete emthod  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```

```
public interface Clarusway {  
  
    public void kursKayit(); // abstract method  
  
    public static default String offer() { // concrete emthod  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```



Interface'lerde Body'si Olan Method Yazılabilir Mi?

TRICK :)

Concrete method'da **“default method”** obje ile **“static method”** method name ile call edilir.

```
public interface Clarusway {  
    public default String offer() { // concrete method  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```



```
public class QaTester implements Clarusway {  
    public static void main(String[] args) {  
        QaTester tester = new QaTester();  
        System.out.println(tester.offer()); //obj ile method call-> Clarusway'de bolcana offer :)  
    }  
}
```

```
public interface Clarusway {  
    public static String offer() { // concrete method  
        return "Clarusway'de bolcana offer :)";  
    }  
}
```



```
public class QaTester implements Clarusway {  
    public static void main(String[] args) {  
        QaTester tester = new QaTester();  
        System.out.println(Clarusway.offer()); //method name ile method call-> Clarusway'de bolcana offer :)  
    }  
}
```



THANKS!

Any questions?

HalUk Bilgin | JAVA Backend Developer

BAŞARI GAYRETE AŞIKTIR 😊

